

Chap 4

Linked List

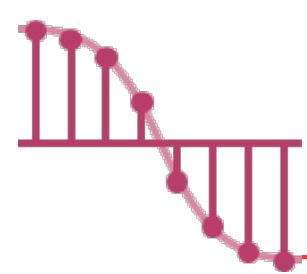
海大資訊工程系

蔡東佐



Outline

- Singly Linked Lists and Chains
- Representing Chains in C
- Linked Stacks and Queues
- Polynomials
- Additional List Operations
- Equivalence Classes
- Sparse Matrices
- Doubly Linked Lists



Singly Linked Lists and Chains _{1/4}

➤ We **studied** the representation of simple data structures using an **array** and a **sequential mapping**.
序项映射表

✓ These representations had the property that **successive nodes of the data object** were stored a fixed distance.

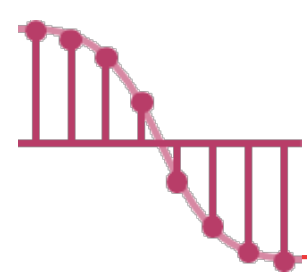
➤ Problems: When a sequential mapping is used for ordered lists:

✓ No more storage available.

✓ We could waste storage.

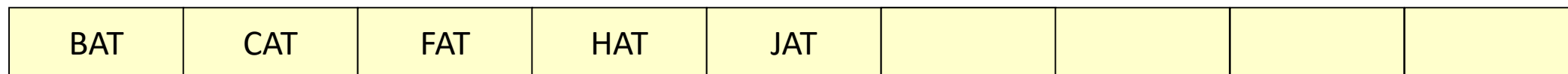
✓ **Excessive data movement is required for deletions and insertions.**



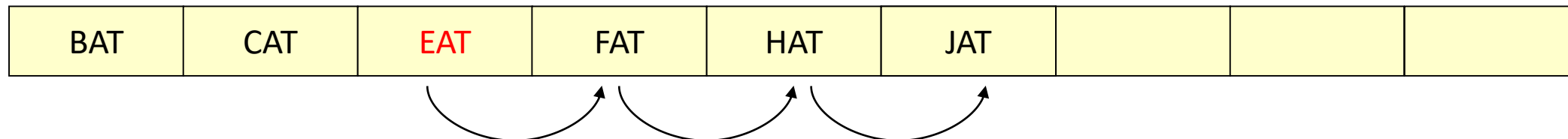


Singly Linked Lists and Chains _{2/4}

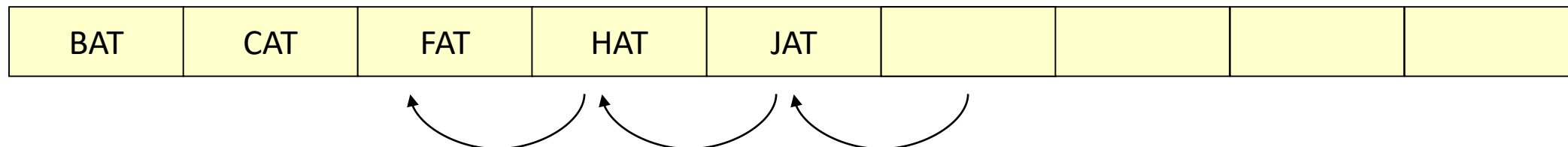
➤ Original

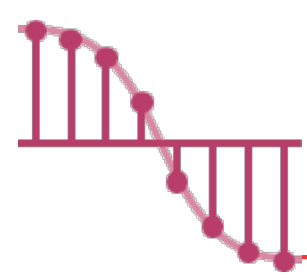


➤ Insert “EAT” after CAT: **you need to move 3 elements**



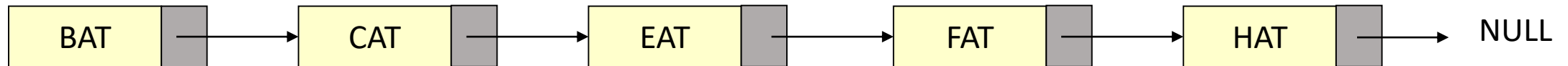
➤ Delete “EAT” : **you need to move 3 elements again!**





Singly Linked Lists and Chains ^{3/4}

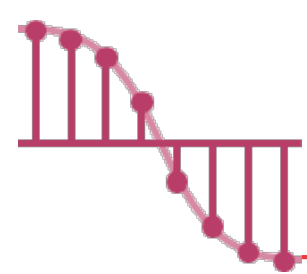
➤ Solution: using **linked** representation



➤ A **linked list** is comprised of nodes; each node has **zero or more data fields** and **one or more link or pointer fields**.

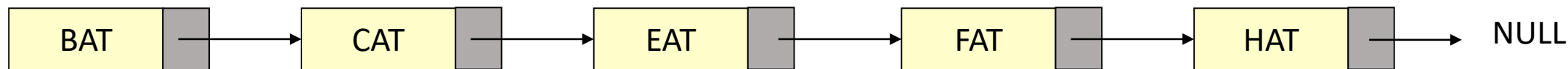
- ✓ The nodes may be placed anywhere in memory.
- ✓ With each node we store the address of location of the next node in the list.

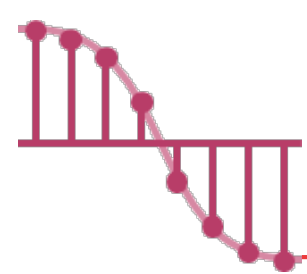




Singly Linked Lists and Chains _{4/4}

- In a **singly linked list**, each node has **exactly one pointer field**.
- A singly linked list in which the **last node has a null link** is called a **chain**.

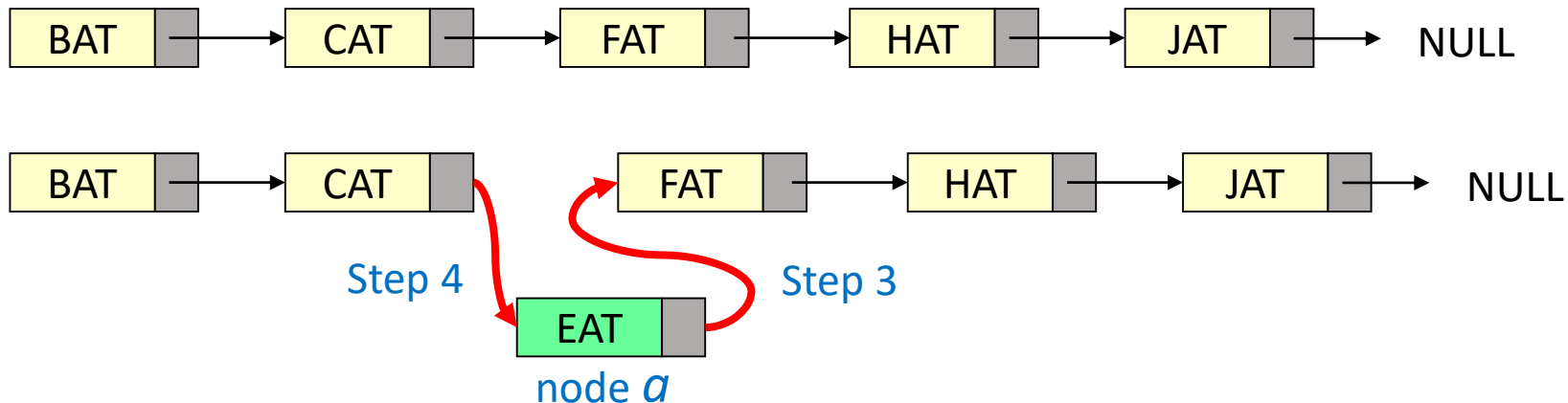


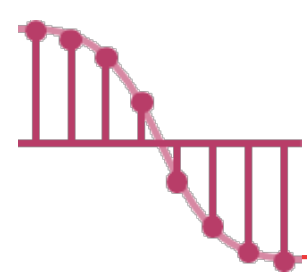


Functions of Linked List _{1/2}

➤ Insert

- ✓ Step 1: Get a node a that is currently unused.
- ✓ Step 2: Set the data field of a to **EAT**.
- ✓ Step 3: Set the link field of a to the node after CAT, which contains FAT.
- ✓ Step 4: Set the link field of the node containing CAT to a .

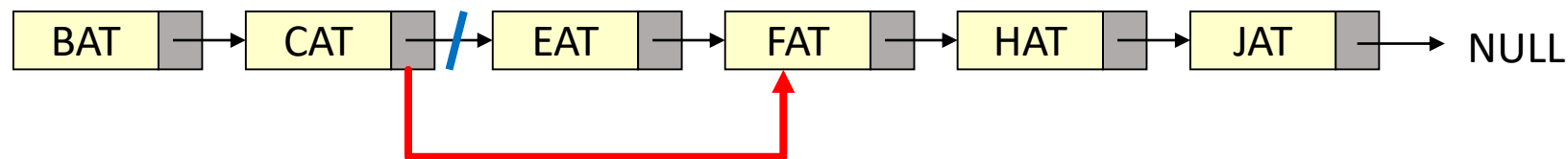


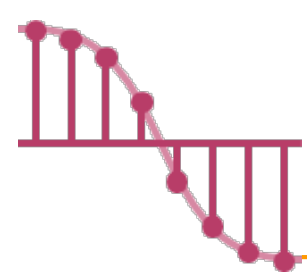


Functions of Linked List _{2/2}

➤ Delete

- ✓ We only need to find the element that immediately precedes EAT, which is CAT, and set its link field to point to EAT's link.
- ✓ We have not moved any data, although the link field of EAT still points to FAT, EAT is no longer in the list.

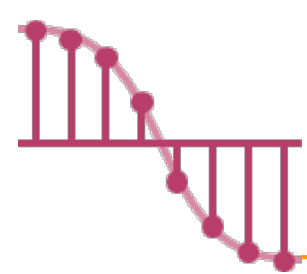




Recall – Pointers

- C provides extensive supports for pointers.
- **Any type T in C there is a corresponding type pointer-to- T .**
- The two most important operators used with the pointer type are:
 - ✓ **& the address operator**
 - ✓ *** the dereferencing (indirection) operator**
- Example:
 - ✓ `int i, *pi;` i is an integer variable and pi is a pointer to an integer.
 - ✓ `pi = &i;` `&i` returns the address of i and assigns it as the value of pi .
 - ✓ `i = 10;` or `*pi = 10;` to assign a value to i
 - ✓ `if (pi == NULL)` or `if (!pi);` to test if the pointer is null.





Using Dynamically Allocated Storage

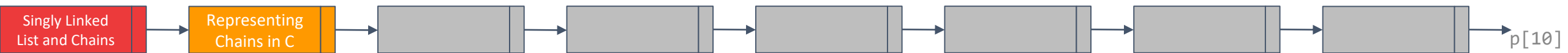
➤ C provides a mechanism, called **heap**, for allocating storage at run-time.

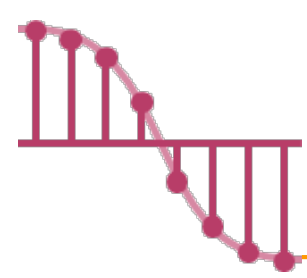
✓ **malloc**: a function provided in C for performing **dynamic memory allocation**.

✓ **free**: a function provided in C for **returning the area of memory** to the system.

➤ Example:

```
int i, *pi;
float f, *pf;
pi = (int *) malloc(sizeof(int));
pf = (float *) malloc(sizeof(float));
*pi = 1024;
*pf = 3.14;
printf("an integer = %d, a float = %f\n", *pi, *pf);
free(pi);
free(pf);
```





Structures 1/3

➤ Declare a variable named **person** that has two fields:

```
struct {  
    char name[4];  
    int age;  
} person;
```

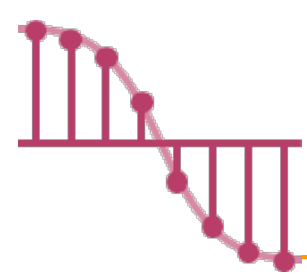
```
strcpy(person.name, "BAT");  
person.age = 10;
```

➤ Declare a structure named **employee** that has two fields:

```
struct employee {  
    char name[4];  
    int age;  
};  
struct employee person;
```

```
strcpy(person.name, "BAT");  
person.age = 10;
```





Structures ^{2/3}

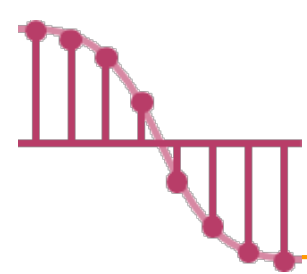
➤ Declare a structure named **employee** and a variable named **person**:

```
struct employee {  
    char name[4];  
    int age;  
} person;
```

```
strcpy(person.name, "BAT");  
person.age = 10;
```

- ✓ struct employee person2, person3; /* 宣告另外兩個變數 */
- ✓ struct employee *link; /* 宣告一個指標, 指到的型態為 struct employee */





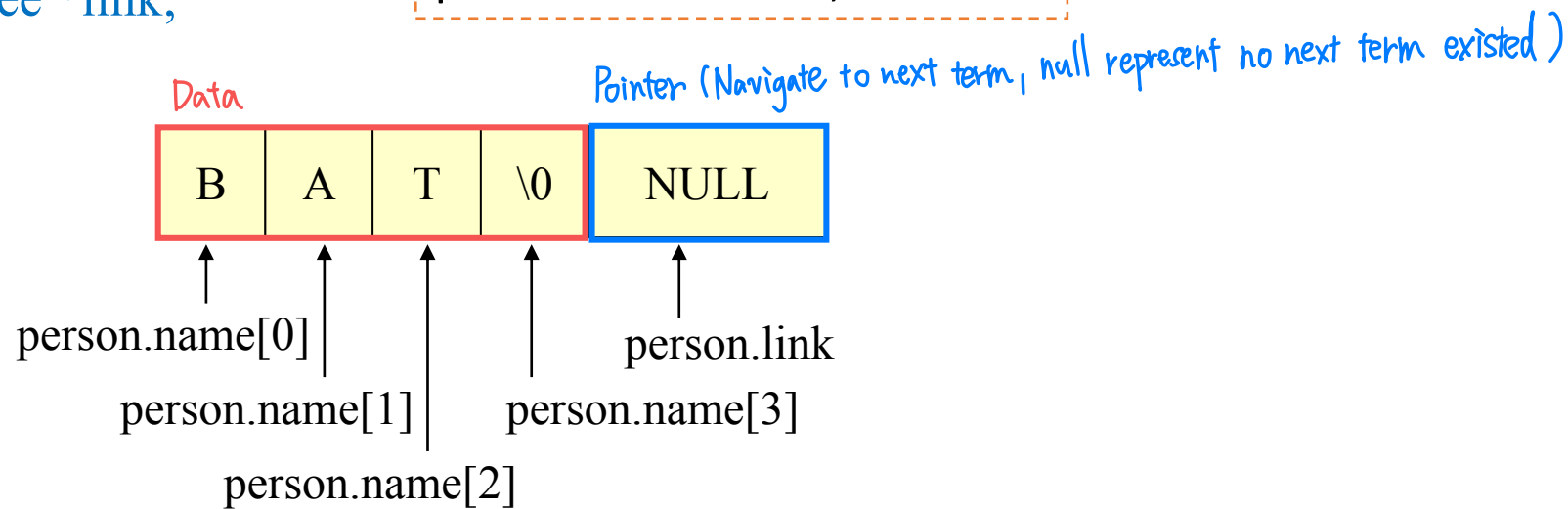
Structures 3/3

➤ Declare a structure named **employee** and a variable named **person**:

- ✓ 一個長度為四的字元陣列
- ✓ 一個指標, 指到的型態為 struct employee

```
struct employee {  
    char name[4];  
    struct employee *link;  
} person;
```

```
strcpy(person.name, "BAT");  
person.link = NULL;
```



Typedef _{1/2}

➤ Declare the other two variables, **person1** and **person2**

✓ `struct employee person1, person2;`



```
struct employee {  
    char name[4];  
    struct employee *link;  
};
```

➤ 使用 typedef 將 “struct employee” 型態名稱縮短

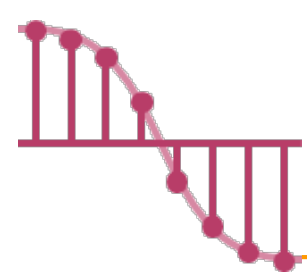
typedef `struct employee` **human;**

human `person1, person2;`

typedef `struct employee` ***humanPointer;**

humanPointer link;
a pointer





Typedef _{2/2}

```
struct employee {  
    char name[4];  
    struct employee *link;  
};
```

= typedef struct employee human;

+

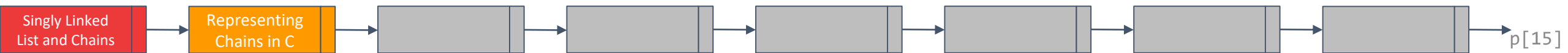
```
struct employee {  
    char name[4];  
    human *link;  
};
```

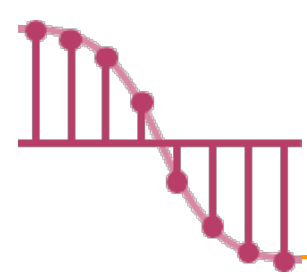
= typedef struct employee * humanPointer;

+

```
struct employee {  
    char name[4];  
    humanPointer link;  
};
```

a pointer





Example 4.1: List of words _{1/2}

➤ The following declarations contain an example of **self-referential structure**.

✓ C allows us to **create a pointer** to a **type that does not yet exist**.

```
typedef struct listNode *listPointer;  
struct listNode{  
    char data[4];  
    listPointer link;  
}
```

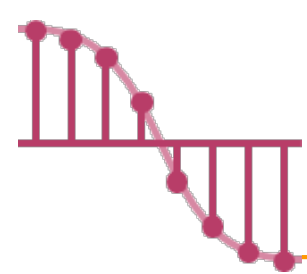
➤ To create a new empty list:

```
listPointer first = NULL;
```

➤ To define an IS_EMPTY macro to test for an empty list:

```
#define IS_EMPTY (first) (!(first))
```





Example 4.1: List of words _{2/2}

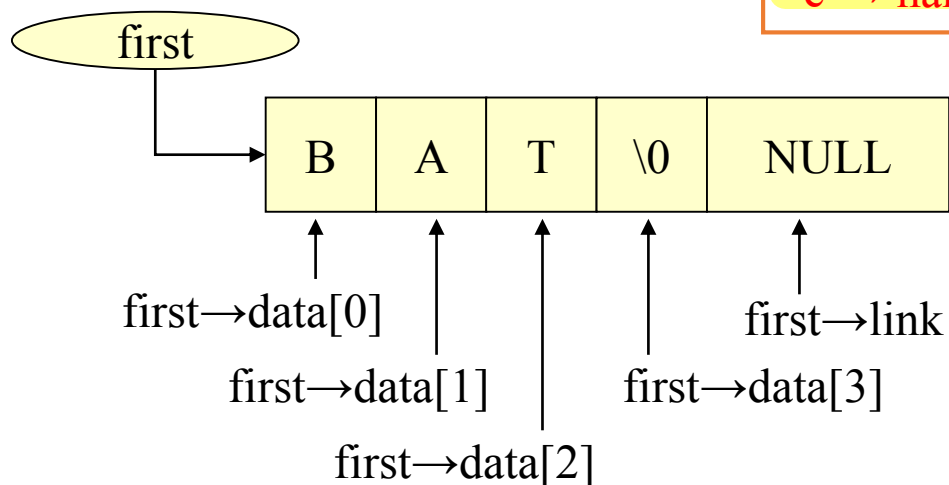
➤ To obtain a new node:

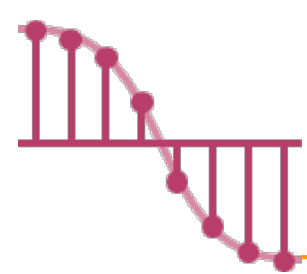
```
first = (listPointer) malloc(sizeof(*first));
```

➤ To place the word BAT into our list:

```
strcpy (first→data, "BAT");  
first→link = NULL;
```

If *e* is a pointer to a structure, then
“*e* → name” is equal to “(**e*).name”



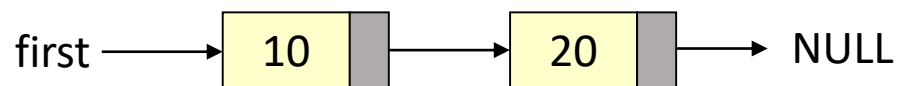


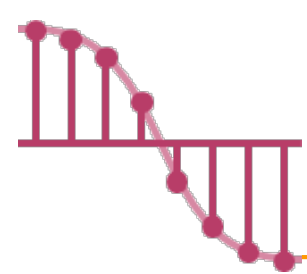
Program 4.1: Create a two-node list

➤ Two-node linked list

```
listPointer create2()
{
    /* create a linked list with two nodes */
    listPointer first, second;
    first = (listPointer)malloc(sizeof(*first));
    second = (listPointer)malloc(sizeof(*second));
    second→link = NULL;
    second→data = 20;
    first→data = 10;
    first→link = second;
    return first;
}
```

```
typedef struct listNode *listPointer;
struct listNode {
    int data;
    listPointer link;
};
```





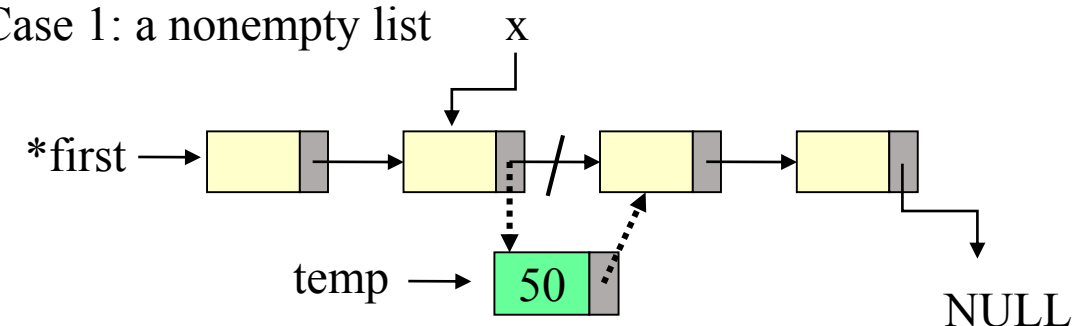
Program 4.2: Simple insert into front of list

➤ List insertion

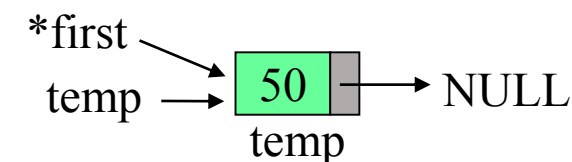
```
void insert(listPointer *first, listPointer x)
{
    /* insert a new node with data = 50 into the chain first after node x */
    listPointer temp;
    temp = (listPointer)malloc(sizeof(*temp));
    if(IS_FULL(temp)){
        printf("The memory is full\n");
        exit(1);
    }
    temp->data = 50;
    if(*first) {           //Case 1: nonempty list
        temp->link = x->link;
        x->link = temp;
    }
    else {                 //Case 2: empty list
        temp->link = NULL;
        *first = temp;
    }
}
```

x : points to the node that precedes the new node we wish to insert.

Case 1: a nonempty list



Case 2: an empty list



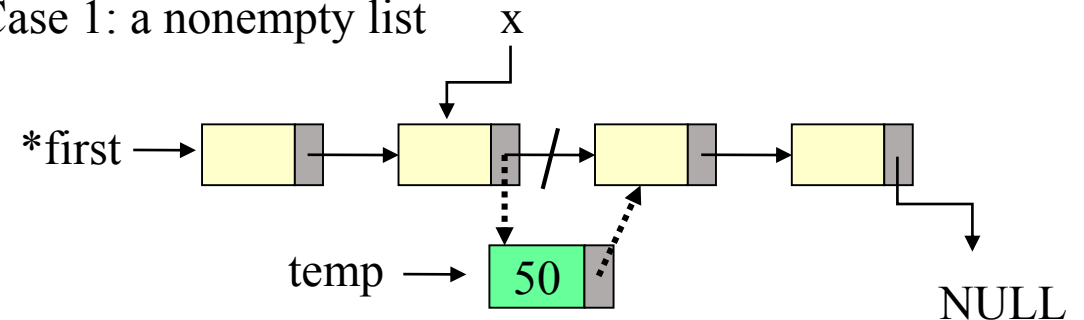
Program 4.2: Simple insert into front of list

➤ List insertion

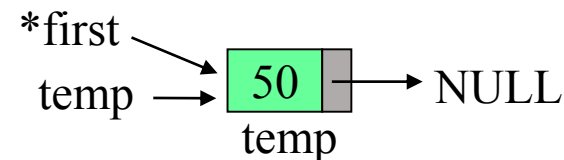
```
void insert(listPointer *first, listPointer x)
{
    /* insert a new node with data = 50 into the chain first after node x */
    listPointer temp;
    temp = (listPointer)malloc(sizeof(*temp));
    if(IS_FULL(temp)){
        printf("The memory is full\n");
        exit(1);
    }
    temp->data = 50;
    if(*first) {                //Case 1: nonempty list
        temp->link = x->link;
        x->link = temp;
    }
    else {                     //Case 2: empty list
        temp->link = NULL;
        *first = temp;
    }
}
```

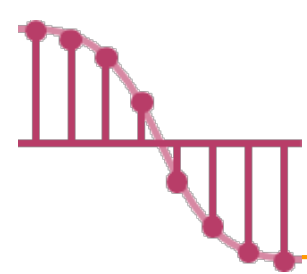
x : points to the node that precedes the new node we wish to insert.

Case 1: a nonempty list



Case 2: an empty list





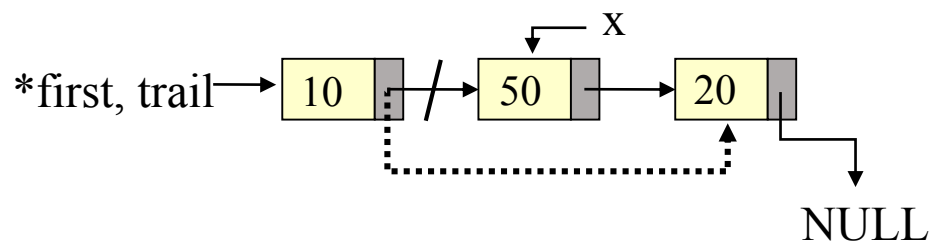
Program 4.3: Deletion from a list

➤ List deletion

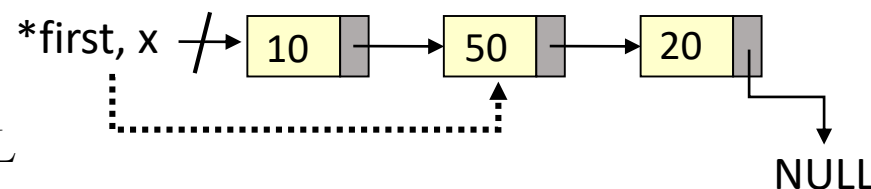
```
void delete(listPointer *first, listPointer trail, listPointer x)
{
    /* delete x from the list, trail is the preceding node and *first is the front of the list */
    if(trail)        // Case 1
        trail→link = x→link;
    else              // Case 2
        *first = (*first)→link;
    free(x);
}
```

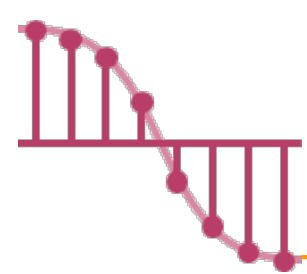
x : points to the node that we wish to delete
trail: points to the node that precedes x

Case 1: trail ≠ NULL



Case 2: trail = NULL





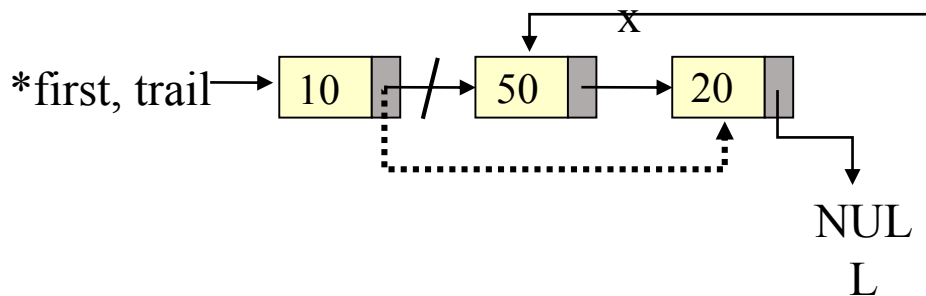
Program 4.3: Deletion from a list

➤ List deletion

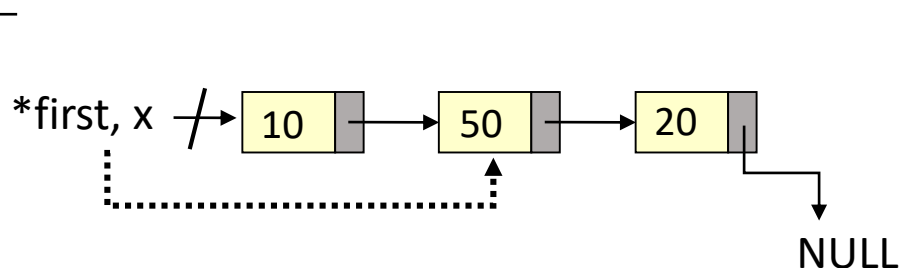
```
void delete(listPointer *first, listPointer trail, listPointer x)
{
    /* delete x from the list, trail is the preceding node and *first is the front of the list */
    if(trail)        // Case 1
        trail→link = x→link;
    else              // Case 2
        *first = (*first)→link;
    free(x);
}
```

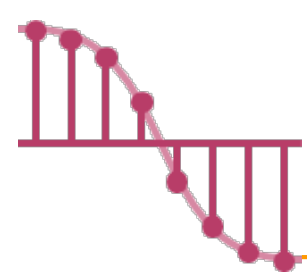
x : points to the node that we wish to delete
trail: points to the node that precedes x

Case 1: trail ≠ NULL



Case 2: trail = NULL

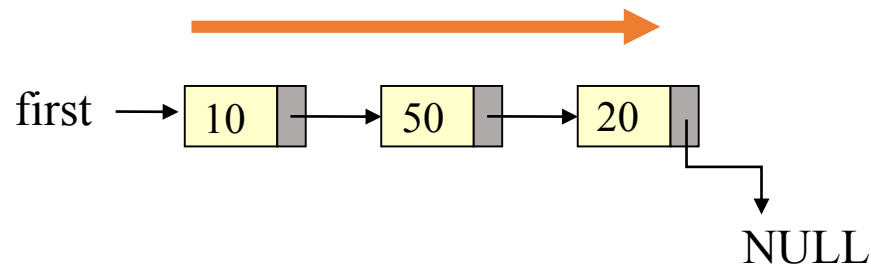


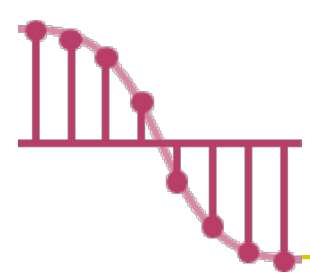


Program 4.4: Printing a list

➤ Printing out a list

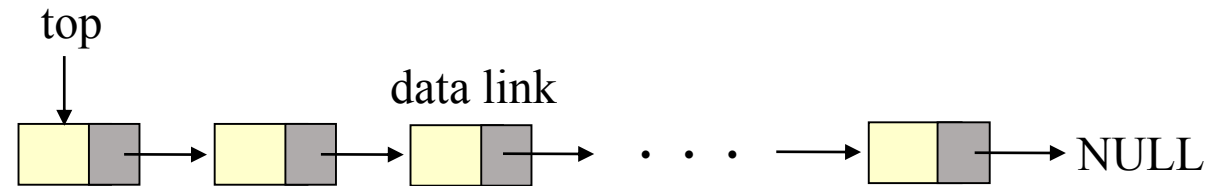
```
void printList(listPointer first)
{
    printf("The list contains: ");
    for ( ; first ; first = first→link )
        printf("%4d", first→data);
    printf("\n");
}
```



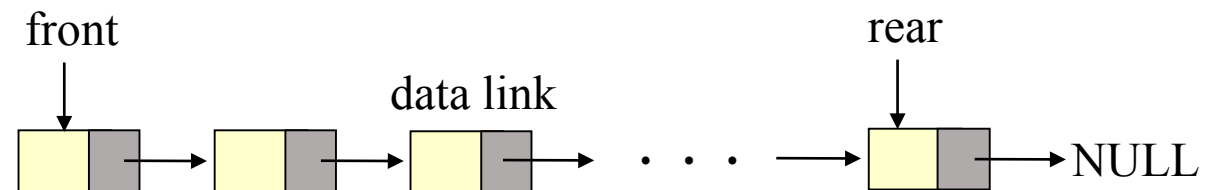


Linked Stacks and Queues

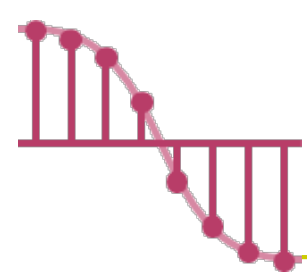
- The **direction of links** for both the **stack** and the **queue** facilitate **easy insertion and deletion of nodes**.
 - ✓ Easily **add or delete a node** from **the top of the stack**.
 - ✓ Easily **add a node** to **the rear of the queue** and **delete a node** at **the front of a queue**.



(a) Linked stack



(b) Linked queue



Stacks – Declarations & Initial Condition

- The declarations (We wish to represent $n \leq 10$ MAX_STACKS **stacks**)

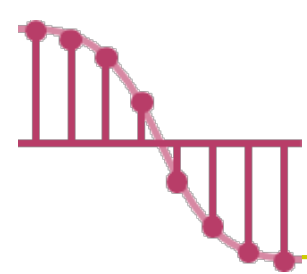
```
#define MAX_STACKS 10 /*maximum number of stacks*/
typedef struct {
    int key;
    /*other fields */
} element;

typedef struct stack *stackPointer;
struct stack {
    element data;
    stackPointer link;
};
stackPointer top[MAX_STACKS];
```

- The initial condition

```
top[i] = NULL;
```

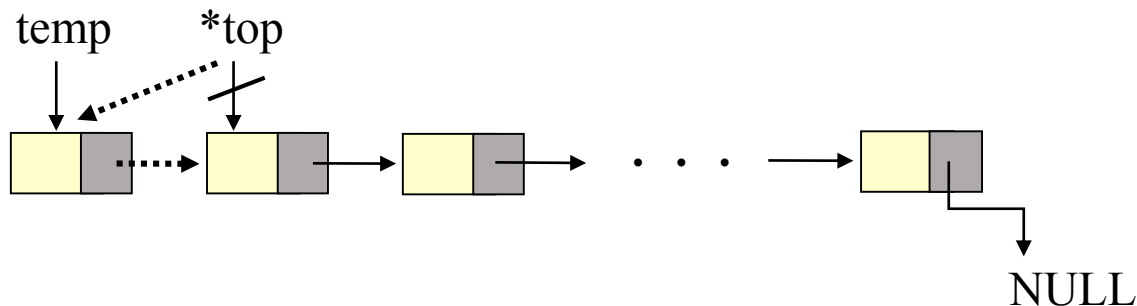


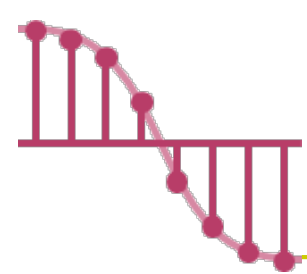


Stacks – Program 4.5

➤ Add to a Linked Stack

```
void push(int i, element item)
{
    /* add item to the i-th stack */
    stackPointer temp;
    MALLOC(temp, sizeof(*temp));
    temp->data = item;
    temp->link = top[i];
    top[i] = temp;
}
```

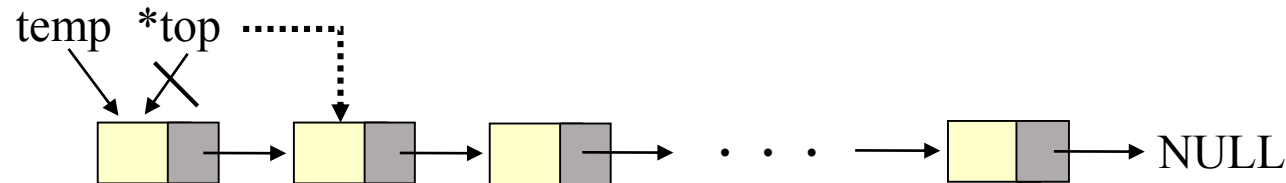


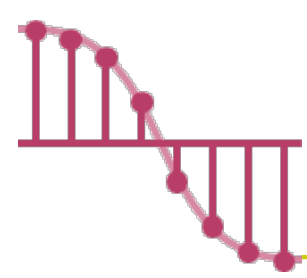


Stacks – Program 4.6

➤ Delete from a Linked Stack

```
element pop(int i)
{
    /* remove top element from the i-th stack */
    stackPointer temp = top[i];
    element item;
    if (!temp)
        return stackEmpty();
    item = temp→data;
    top[i] = temp→link;
    free(temp);
    return item;
}
```





Queues – Declarations & Initial Condition

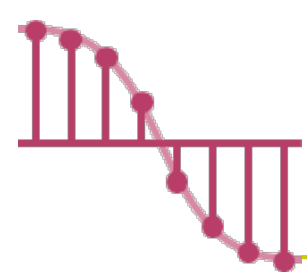
- The declarations (We wish to represent $n \leq 10$ MAX_STACKS **queues**)

```
#define MAX_QUEUES 10 /*maximum number of queues*/  
typedef struct queue *queuePointer;  
  
struct queue {  
    element data;  
    queuePointer link;  
};  
queuePointer front[MAX_QUEUES], rear[MAX_QUEUES];
```

- The initial condition

$\text{front}[i] = \text{NULL}, \text{rear}[i] = \text{NULL};$



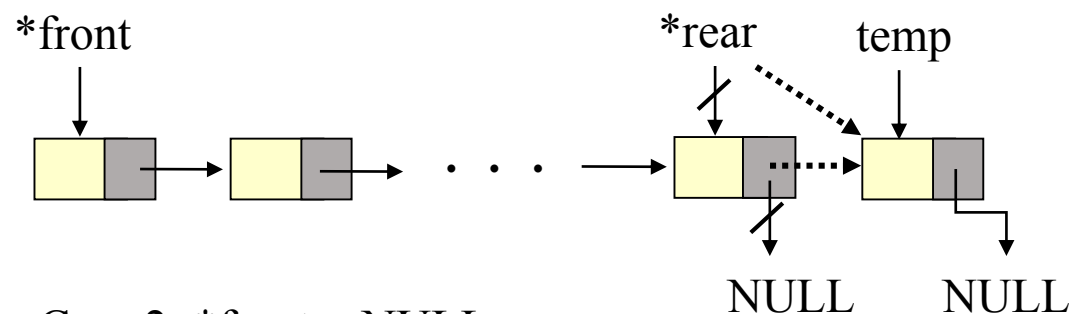


Queues – Program 4.7

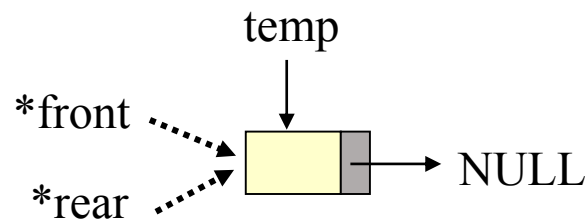
➤ Add to the rear of a linked queue

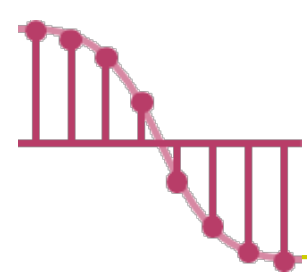
```
void addq(i, item)
{
    /* add item to the rear of queue i */
    queuePointer temp;
    MALLOC(temp, sizeof(*temp));
    temp→data = item;
    temp→link = NULL;
    if (front[i]) 非首项
        rear[i]→link = temp;
    else 首项
        front[i] = temp;
    rear[i] = temp;
}
```

Case 1: $*front \neq NULL$



Case 2: $*front = NULL$

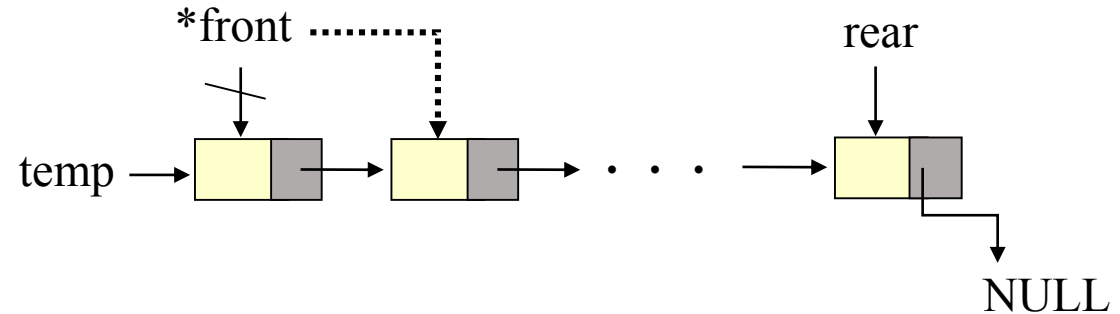


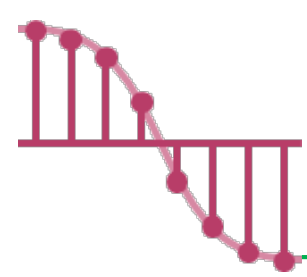


Queues – Program 4.8

➤ Delete from the front of a linked queue

```
element deleteq(int i)
{
    /* delete an element from queue i */
    queuePointer temp = front[i];
    element item;
    if (!temp)
        return queueEmpty();
    item = temp→data
    front[i] = temp→link;
    free(temp);
    return item;
}
```





Polynomial Representation

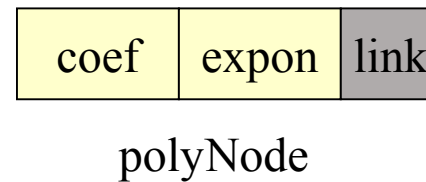
➤ We want to represent the polynomial :

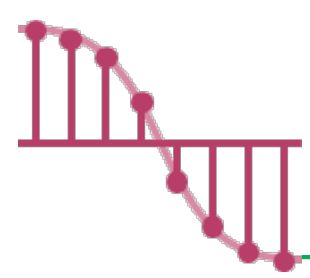
$$A(x) = a_{m-1}x^{e_{m-1}} + \dots + a_0 x^{e_0}$$

➤ We will represent each term as a node containing **coefficient** and **exponent** fields, as well as a **pointer** to the next term.

➤ Declarations:

```
typedef struct polyNode *polyPointer;  
struct polyNode {  
    int coef;  
    int expon;  
    polyPointer link;  
};  
polyPointer a, b;
```

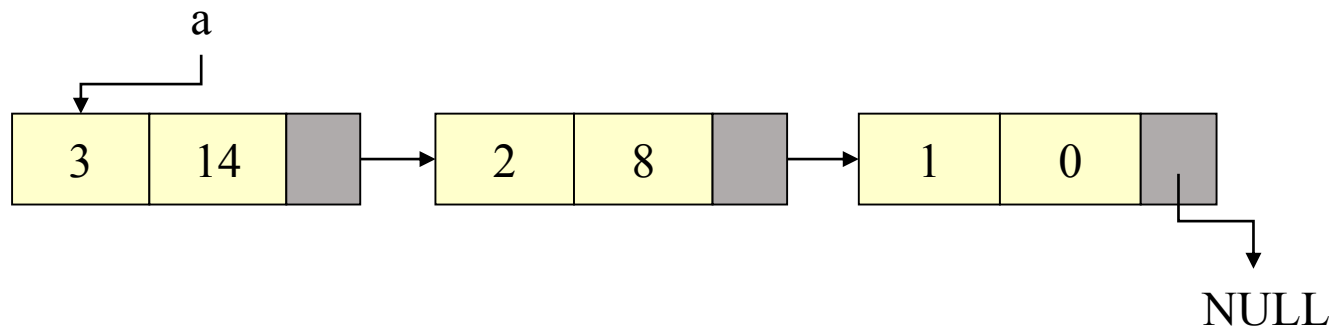




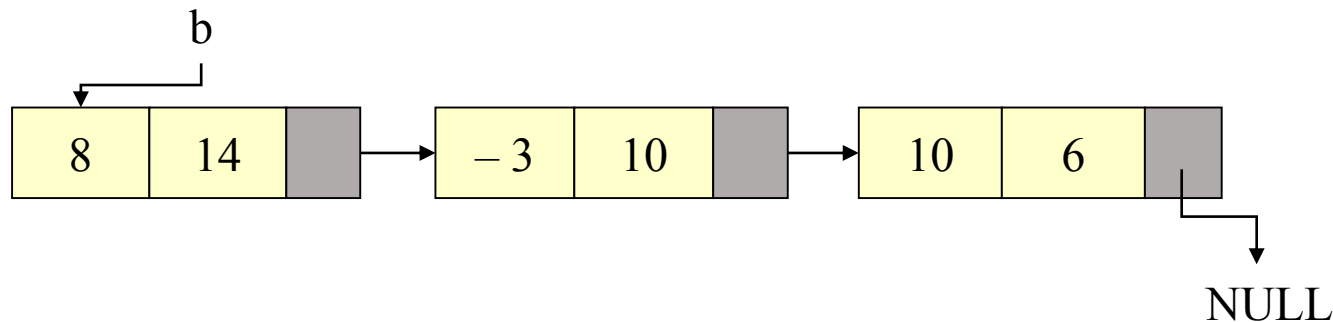
Add Two Polynomials

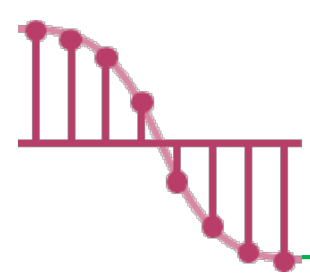
➤ Examples:

✓ $a = 3x^{14} + 2x^8 + 1$



✓ $b = 8x^{14} - 3x^{10} + 10x^6$

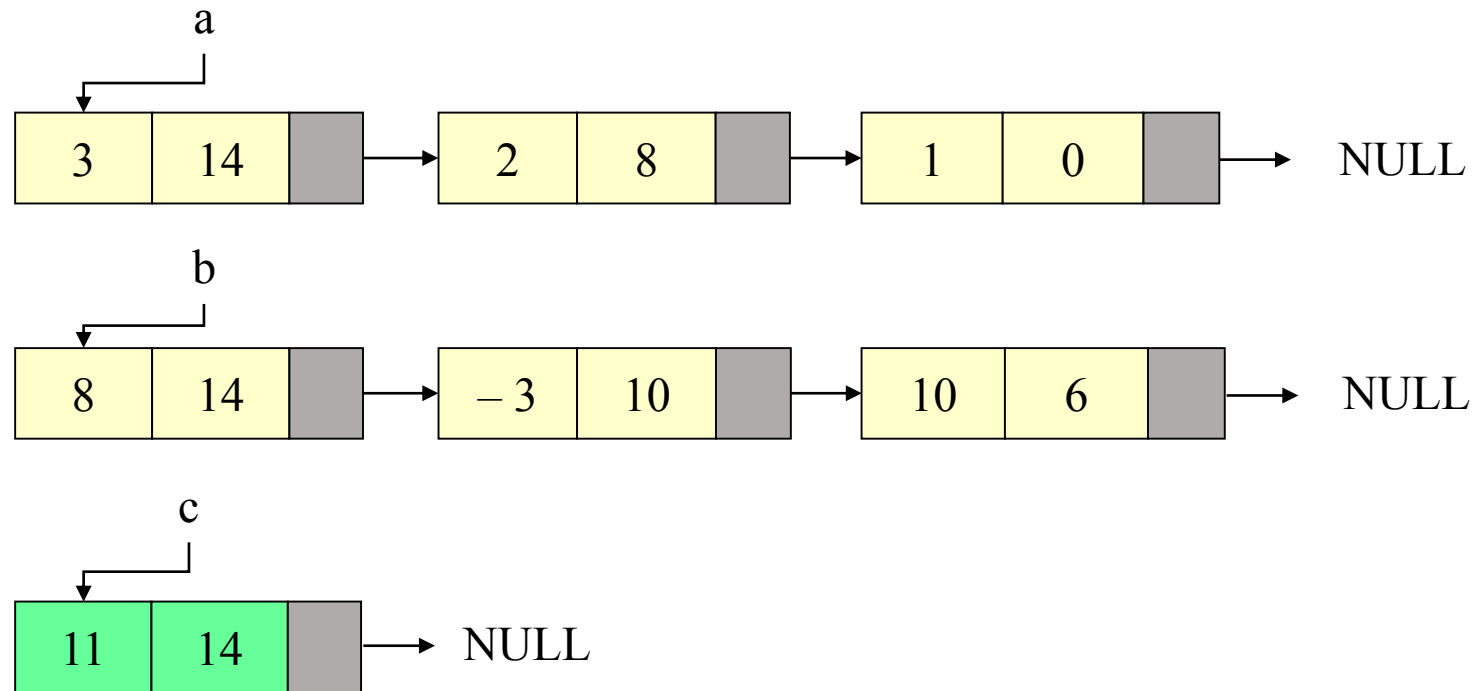


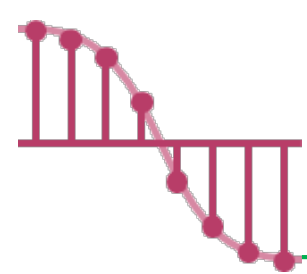


Add Two Polynomials – Algorithm _{1/3}

➤ Generating the first three terms of $c = a + b$

✓ $a \rightarrow \text{expon} == b \rightarrow \text{expon}$

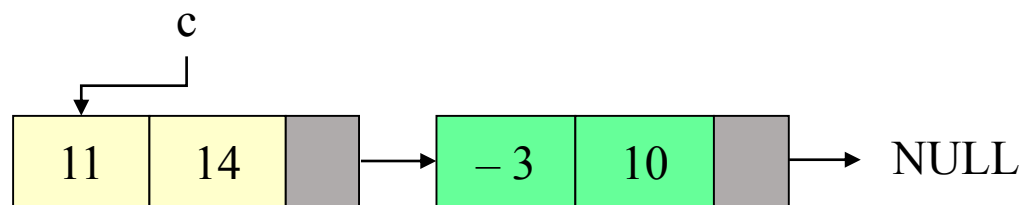
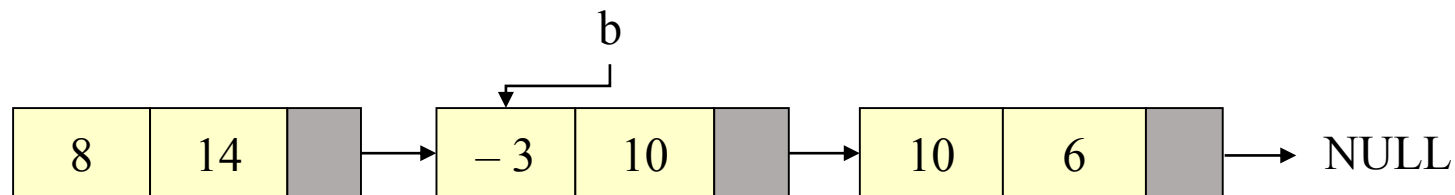
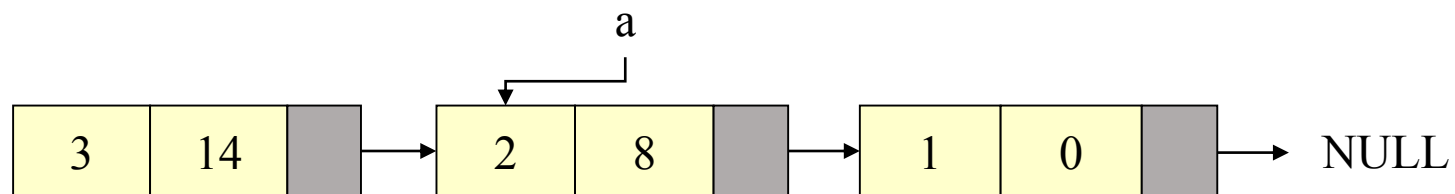


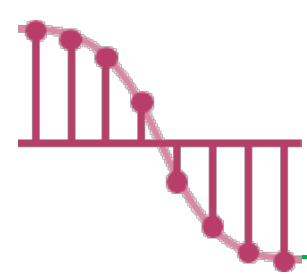


Add Two Polynomials – Algorithm _{2/3}

➤ Generating the first three terms of $c = a + b$

✓ $a \rightarrow \text{expon} < b \rightarrow \text{expon}$

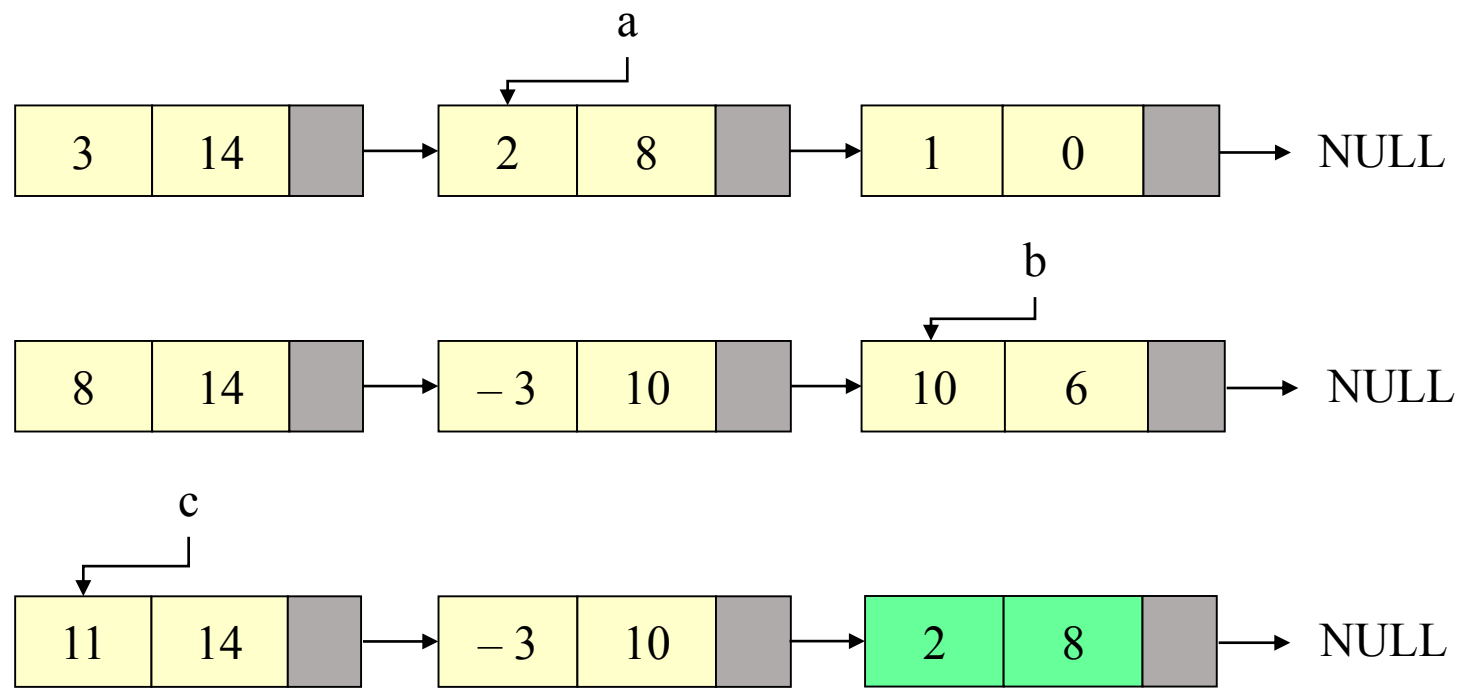




Add Two Polynomials – Algorithm _{3/3}

➤ Generating the first three terms of $c = a + b$

✓ $a \rightarrow \text{expon} > b \rightarrow \text{expon}$



Add Two Polynomials – Program 4.9

```
polyPointer padd(polyPointer a, polyPointer b)
{
    /* return a polynomial which is the sum of a and b */
    polyPointer front, rear, temp;
    int sum;
    rear = (polyPointer)malloc(sizeof(*rear));
    if (IS_FULL(rear)){
        printf("The memory is full\n");
        exit(1)
    }
    front = rear;
    while (a && b)
        switch (COMPARE(a->expon, b->expon)) {
            case -1: /* a->expon < b->expon */
                attach(b->coef, b->expon, &rear);
                b = b->link;
                break;
            case 0: /* a->expon = b->expon */
                sum = a->coef + b->coef;
                if (sum) attach(sum, a->expon, &rear);
                a = a->link; b = b->link; break;
            case 1: /* a->expon > b->expon */
                attach(a->coef, a->expon, &rear);
                a = a->link;
        }
}
```

→ 一直重複，直到 a & b 的尾

/* copy rest of list a and then list b */

```
for (; a; a = a->link) attach(a->coef, a->expon, &rear);
for (; b; b = b->link) attach(b->coef, b->expon, &rear);
rear->link = NULL;
/* delete extra initial node */
temp = front; front = front->link; free(temp);
return front;
}
```

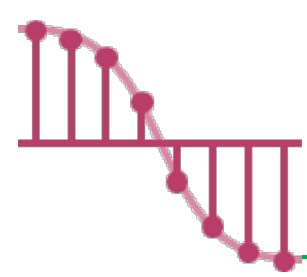
```
void attach(float coefficient, int exponent, polyPointer *ptr)
{

```

/* create a new node with coef = coefficient and
expon = exponent, attach it to the node pointed to by ptr
is updated to point to this new node */

```
polyPointer temp;
temp = (polyPointer)malloc(sizeof(*temp));
if (IS_FULL(temp)) {
    printf("The memory is full\n");
    exit(1);
}
temp->coef = coefficient;
temp->expon = exponent;
(*ptr)->link = temp;
*ptr = temp;
}
```

→ 負責合併項數



Analysis of padd

➤ Assume that a and b have m and n terms, respectively.

$$A(x) = a_{m-1}x^{e_{m-1}} + \dots + a_0 x^{e_0} \quad B(x) = b_{n-1}x^{f_{n-1}} + \dots + b_0 x^{f_0}$$

➤ There are three cost measures:

✓ **coefficient additions**

- $0 \leq \text{number of coefficient additions} \leq \min\{m, n\}$

✓ **exponent comparisons**

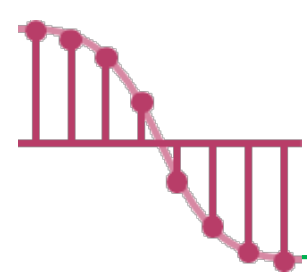
- On each iteration, either a or b or both move to the next term.
- The maximum number of exponent comparisons is $m + n$.

✓ **create of new nodes for c**

- The maximum number of terms in c is $m + n$.

➤ Therefore, the computing time is $O(m + n)$.





Erasing Polynomials

➤ The use of linked lists is well suited to polynomial operations.

✓ Read in polynomials a , b and d and then compute $e = a * b + d$.

```
polyPointer a, b, d, e
```

```
.  
.   
.
```

```
a = readPoly( );
```

```
b = readPoly( );
```

```
d = readPoly( );
```

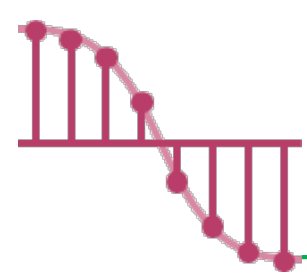
```
temp = pmult(a, b);
```

```
e = padd(temp,d);
```

```
printPoly(e);
```

✓ We created `temp` only to hold a partial result for d .

✓ By returning the nodes of `temp`, we may use them to hold other polynomials.



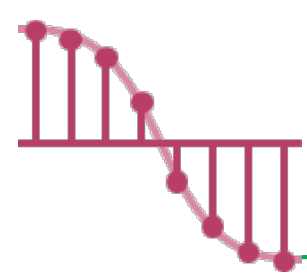
Erasing Polynomials – Program 4.11

➤ **One by one**, erase frees the nodes in *temp*.

```
void erase(polyPointer *ptr)
{
    /* erase the polynomial pointed to by ptr */
    polyPointer temp;
    while (*ptr) {
        temp = *ptr;
        *ptr = (*ptr)→link;
        free(temp);
    }
}
```

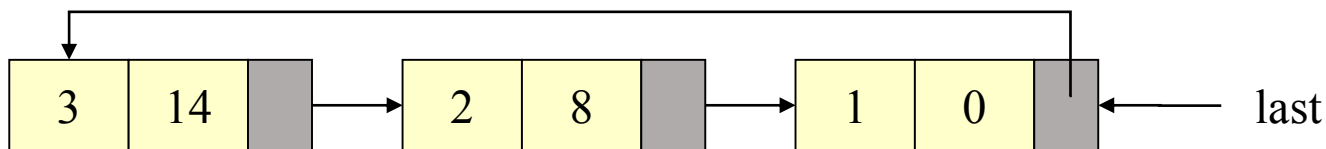
Can we free all the nodes of a polynomial more efficiently ?

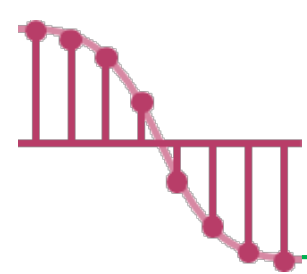




Circular List Representation of Polynomials ^{1/2}

- We modify our list structure so that the link field of **the last node** **points to the first node** in the list.
- ✓ We call this a **circular list**
- ✓ Circular representation of $3x^{14} + 2x^8 + 1$

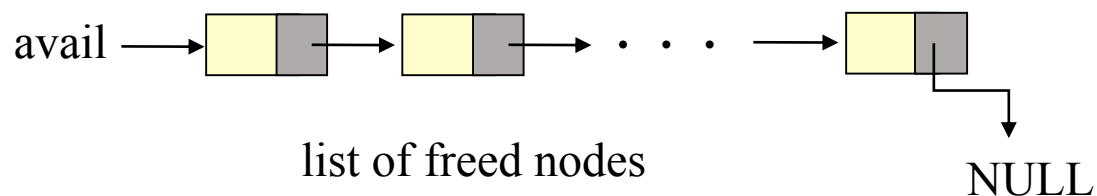




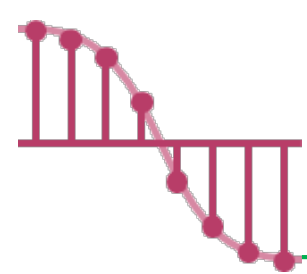
Circular List Representation of Polynomials 2/2

➤ In order to obtain **an efficient erase** algorithm.

- ✓ We **maintain our own list of nodes** that have been “freed”. 不直接清掉! 暫存起來!
- ✓ Only when **the list is empty** do we need to use **malloc** to create a new node.
- ✓ Let **avail** be a variable of type polyPointer that points to **the first node in our list of freed nodes**.

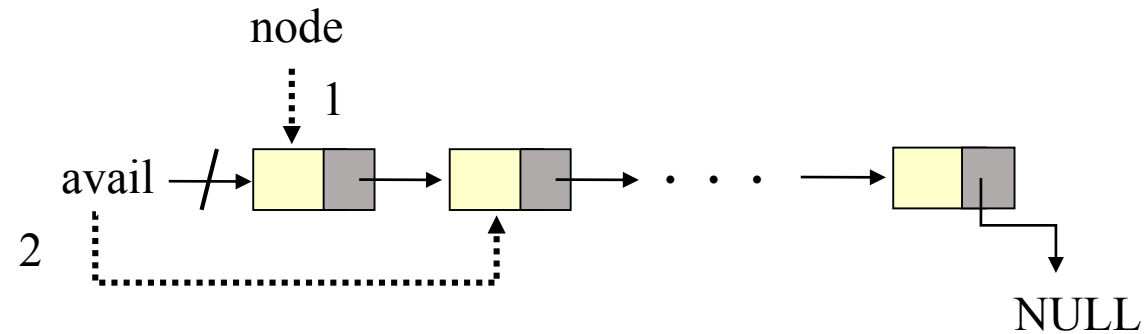


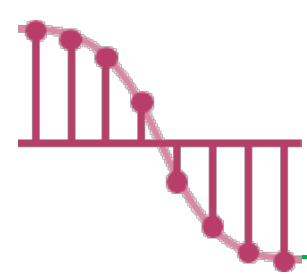
- ✓ Instead of using malloc and free, we now use **getNode** and **retNode**.
- ✓ To avoid the special case of zero polynomials, we introduce a header node into each polynomial.



Program 4.12: getNode function

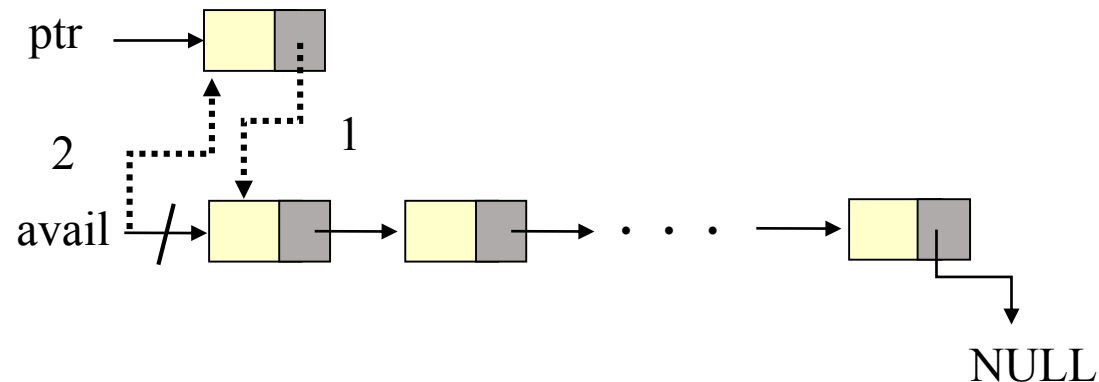
```
polyPointer getNode(void)
{
    /* provide a node for use */
    polyPointer node;
    if (avail) {
        node = avail;
        avail = avail → link;
    }
    else
        MALLOC(node, sizeof(*node));
    return node;
}
```

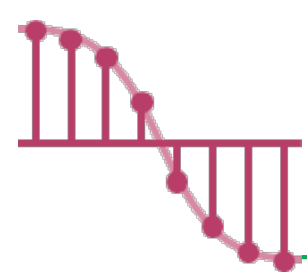




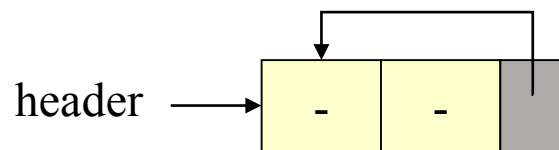
Program 4.13: retNode function

```
void retNode(polyPointer ptr)
{
    /* return a node to the available list */
    ptr->link = avail;
    avail = ptr;
}
```

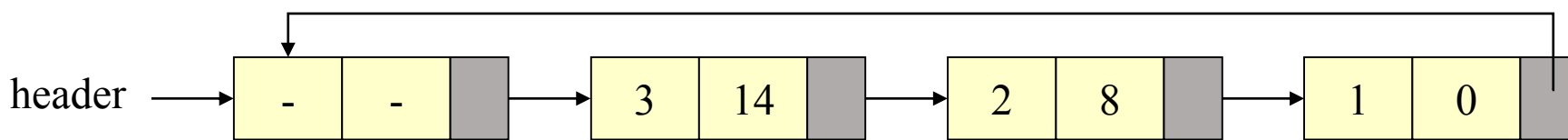




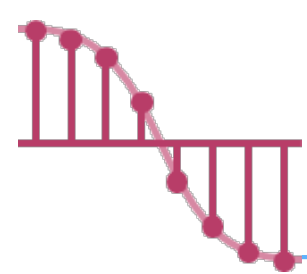
Example: Polynomials with header nodes



(a) Zero polynomial



(b) $3x^{14} + 2x^8 + 1$

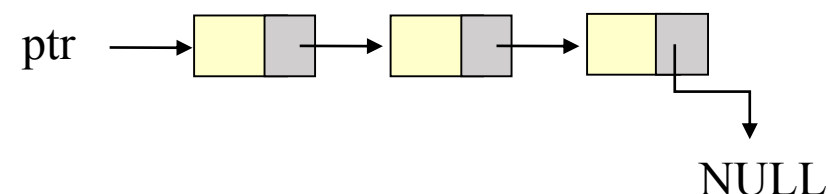


Additional List Operations

➤ It is often necessary, and desirable, **to build a variety of functions** for **manipulating singly linked list**.

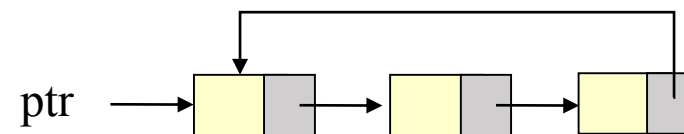
✓ A singly linked list in which the last node has a null link is called **a chain**.

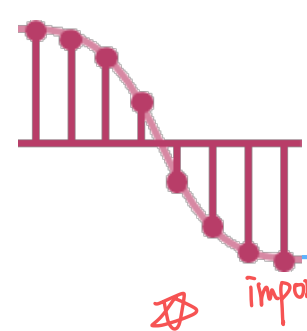
- Operations: **Invert a chain & Concatenate two chains**



✓ A singly linked list in which the link field of the last node points to the first node is called **a circular list**.

- Operations: **Insert a node at the front of a circular list & Determine the length of a circular list**





Program 4.16: Inverting a singly linked list

important!

```
listPointer invert (listPointer lead)
```

```
{
```

```
    /* invert the list pointed to by lead */
```

```
    listPointer middle, trail;
```

```
    middle = NULL;
```

```
    while (lead){
```

```
        trail = middle;
```

```
        middle = lead;
```

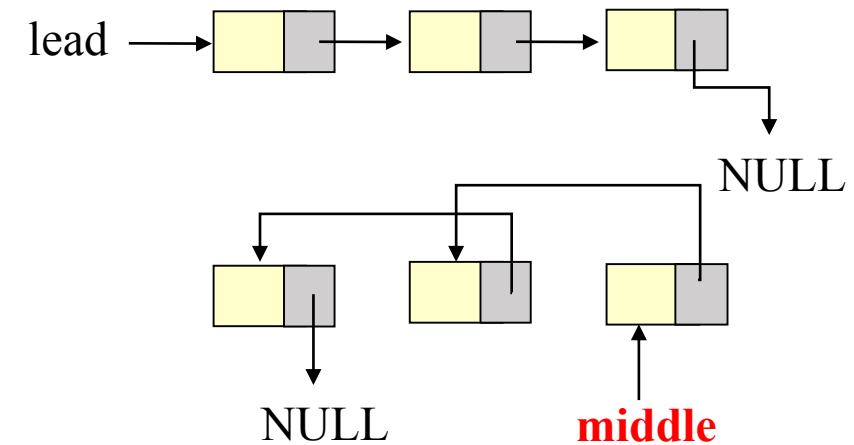
```
        lead = lead->link;
```

```
        middle->link = trail;
```

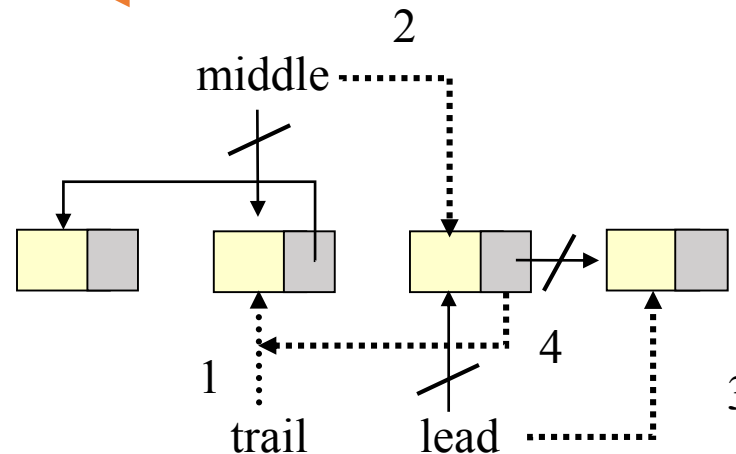
```
    }
```

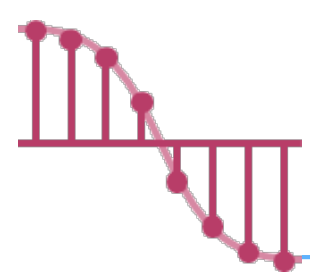
```
    return middle;
```

```
}
```



been inverted





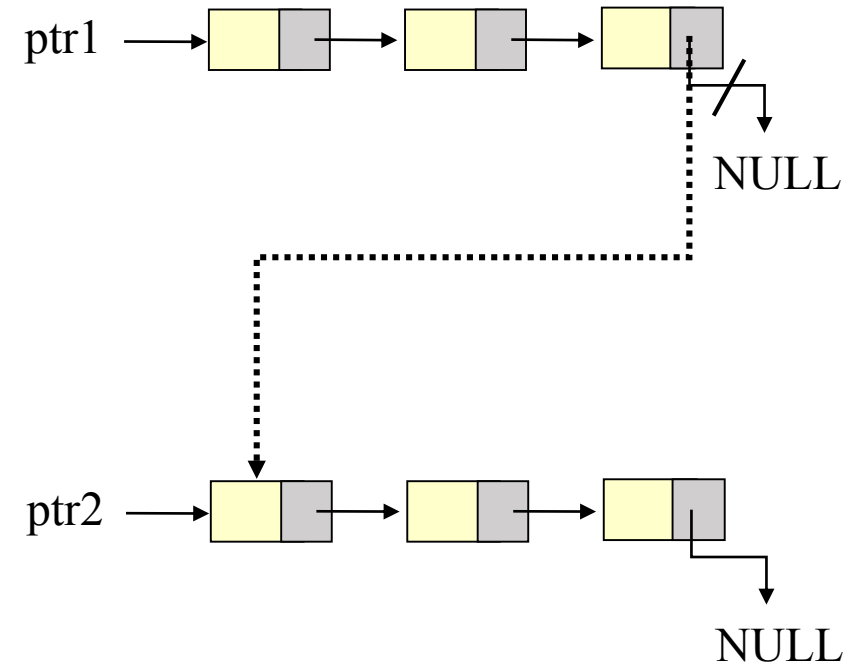
Program 4.17: Concatenating singly linked lists

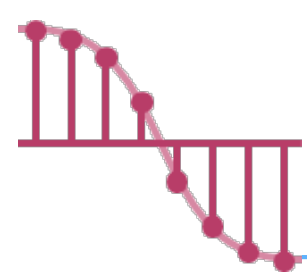
```
listPointer concatenate (listPointer ptr1, listPointer ptr2)
{
    /* produce a new list that contains the list ptr1
    followed by the list ptr2. The list pointed to by ptr1
    is changed permanently */

    listPointer temp;
    /* check for empty lists */
    if (!ptr1) return ptr2;
    if (!ptr2) return ptr1;

    /* neither list is empty, find end of first list */
    for (temp = ptr1; temp->link; temp = temp->link) ;

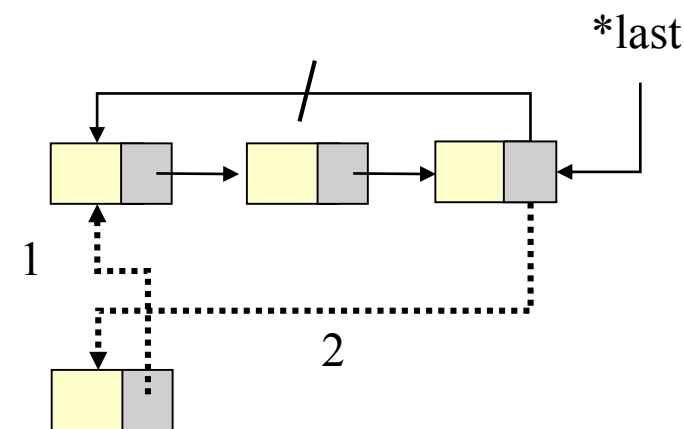
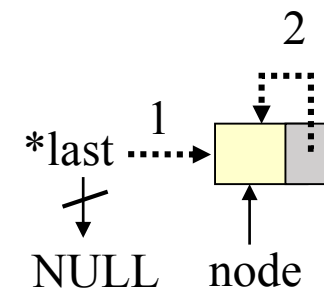
    /* linked end of first to start of second */
    temp->link = ptr2;
}
```

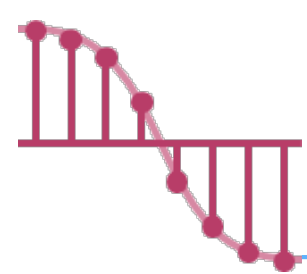




Program 4.18: Inserting at the front of a list

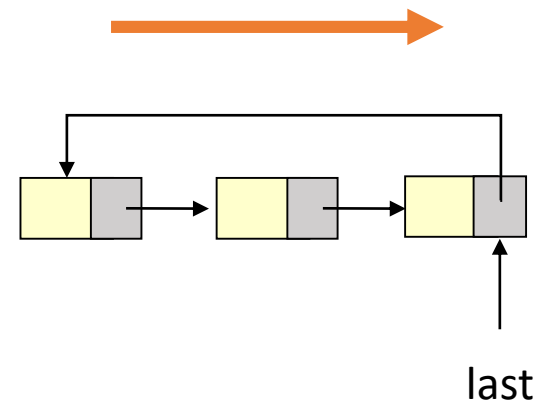
```
void insertFront (listPointer *last, listPointer node)
{
    /* insert node at the front of the circular list
    whose last node is last */
    if (!(*last)){
        /* list is empty, change last to
        point to new entry */
        *last = node;
        node->link = node;
    }
    else {
        /* list is not empty, add new entry
        at front */
        node->link = (*last)->link;
        (*last)->link = node;
    }
}
```

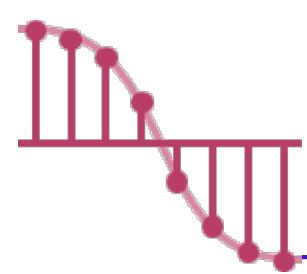




Program 4.19: Finding the length of a circular list

```
int length (listPointer last)
{
    /* find the length of circular list last */
    listPointer temp;
    int count = 0;
    if (last){
        temp = last;
        do{
            count++;
            temp = temp->link;
        } while (temp != last);
    }
    return count;
}
```





Equivalence Classes 等價集合

➤ Let us put together some of the concepts on linked and sequential representation to solve a problem that arises in the design and manufacture of very large-scale integrated (VLSI) circuits.

➤ **Definition:**

✓ A relation over a set, S , is said to be an *equivalence relation* over S iff it is symmetric, reflexive, and transitive over S .

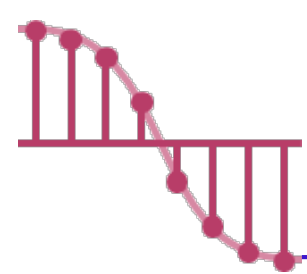
- symmetric, if $x=y$, then $y=x$
- reflexive, $x=x$
- transitive, if $x=y$ and $y=z$, then $x=z$

➤ **Example:**

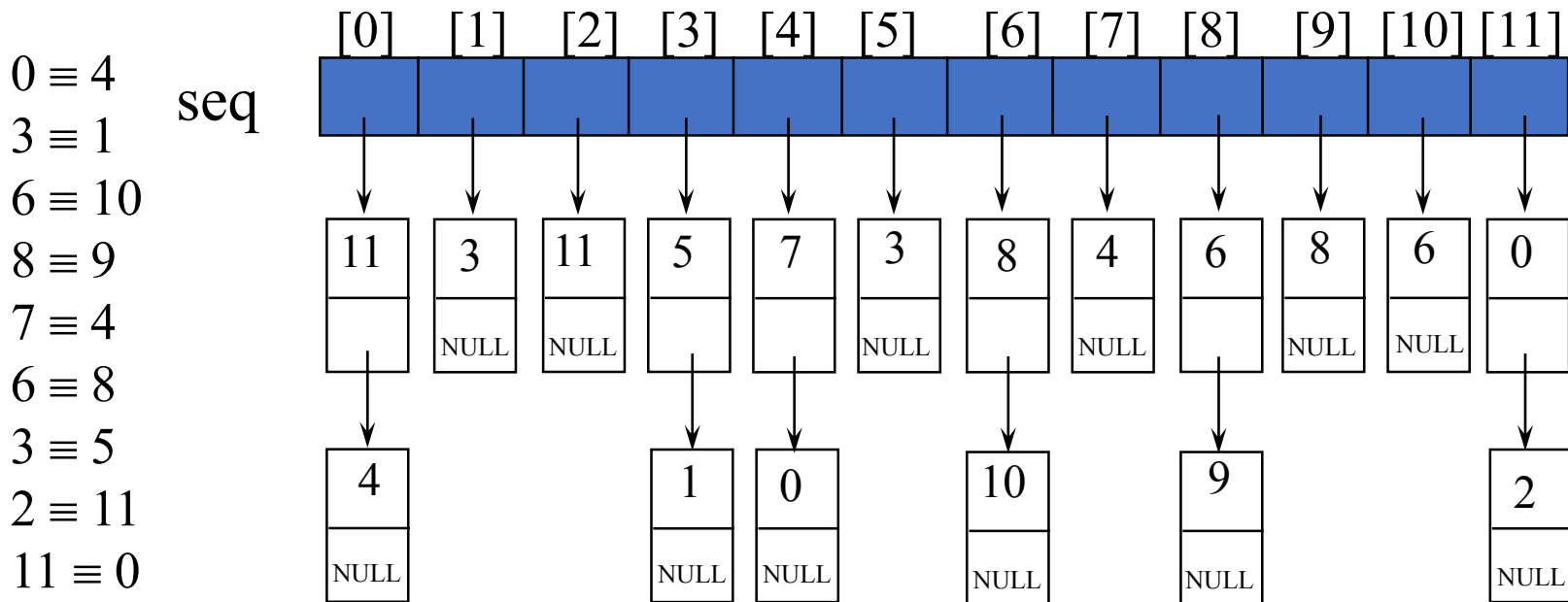
0=4, 3=1, 6=10, 8=9, 7=4,
6=8, 3=5, 2=11, 11=0

three equivalent classes
 $\{0,2,4,7,11\}$ $\{1,3,5\}$ $\{6,8,9,10\}$

```
void equivalence()
{
    read n; // read in number of objects
    initialize seq to 0 and out to FALSE;
    while (more pairs) // input pairs
    {
        read the next pair (i, j);
        put j on the seq[i] list;
        put i on the seq[j] list;
    }
    /*----- print out equivalence classes -----*/
    for (i=0; i<n; i++){
        for( out[i] == FALSE ) {
            out[i] = TRUE;
            output the equivalence class that contains object i;
        }
    }
};
```



Equivalence Classes – Lists after Pairs have been input



$0 \equiv 4$
 $3 \equiv 1$
 $6 \equiv 10$
 $8 \equiv 9$
 $7 \equiv 4$
 $6 \equiv 8$
 $3 \equiv 5$
 $2 \equiv 11$
 $11 \equiv 0$

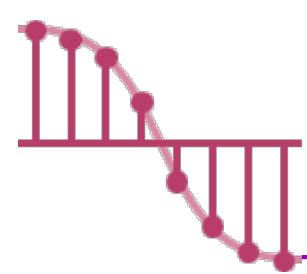
```

typedef struct node *nodePointer ;
typedef struct node {
    int data;
    nodePointer link;
};
    
```

```

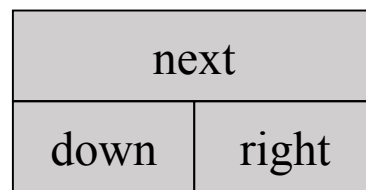
void equivalence () // Phase 2: output equivalence classes
.... (previous page)
for (k=0; k<n; k++){
    if (out[k] == FALSE) { // needs to be output
        cout << endl << "A new class: " << k; out[k] = TRUE;
        ListNode *x = seq[k]; ListNode *top = 0; // init stack
        while (1) { // find rest of class
            while (x) { // process the list
                j = x->data;
                if ( out[j] == FALSE ) {
                    cout << " " << j; out[j] = TRUE;
                    ListNode *y = x->link;
                    x->link = top; top = x; x = y;
                } else x = x->link; // skip current node x
            }
            if ( ! top ) break;
            else { x = seq[top->data]; top = top->link; // unstack }
        } // end of while(1)
    } // end of if (out[k] == FALSE)
}
for ( k=0; k<n; k++)
    while(seq[k]) { ListNode *delnode = seq[k]; seq[k] = delnode->link; delete delnode; }
    delete [] seq; delete [] out;
}
    
```

0 4 11
 x 11 0 0
 y 4 0
 m equivalence pairs
 n nodes
 → O(m+n) algorithm

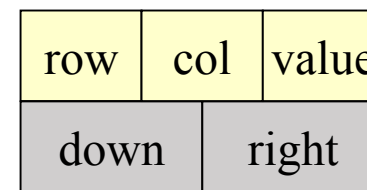


Sparse Matrices

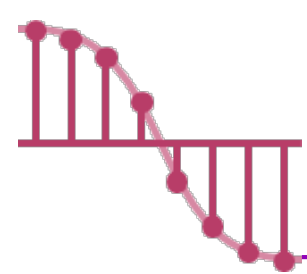
- When we performed matrix operations such as $+$, $-$, or $*$, the number of nonzero terms varied.
- The sequential representation of sparse matrices suffered from the same inadequacies as the similar representation of polynomials.
- We use a linked list representation for sparse matrices.
- There are two types of nodes in the representation: header nodes and elements nodes.



header node



element node

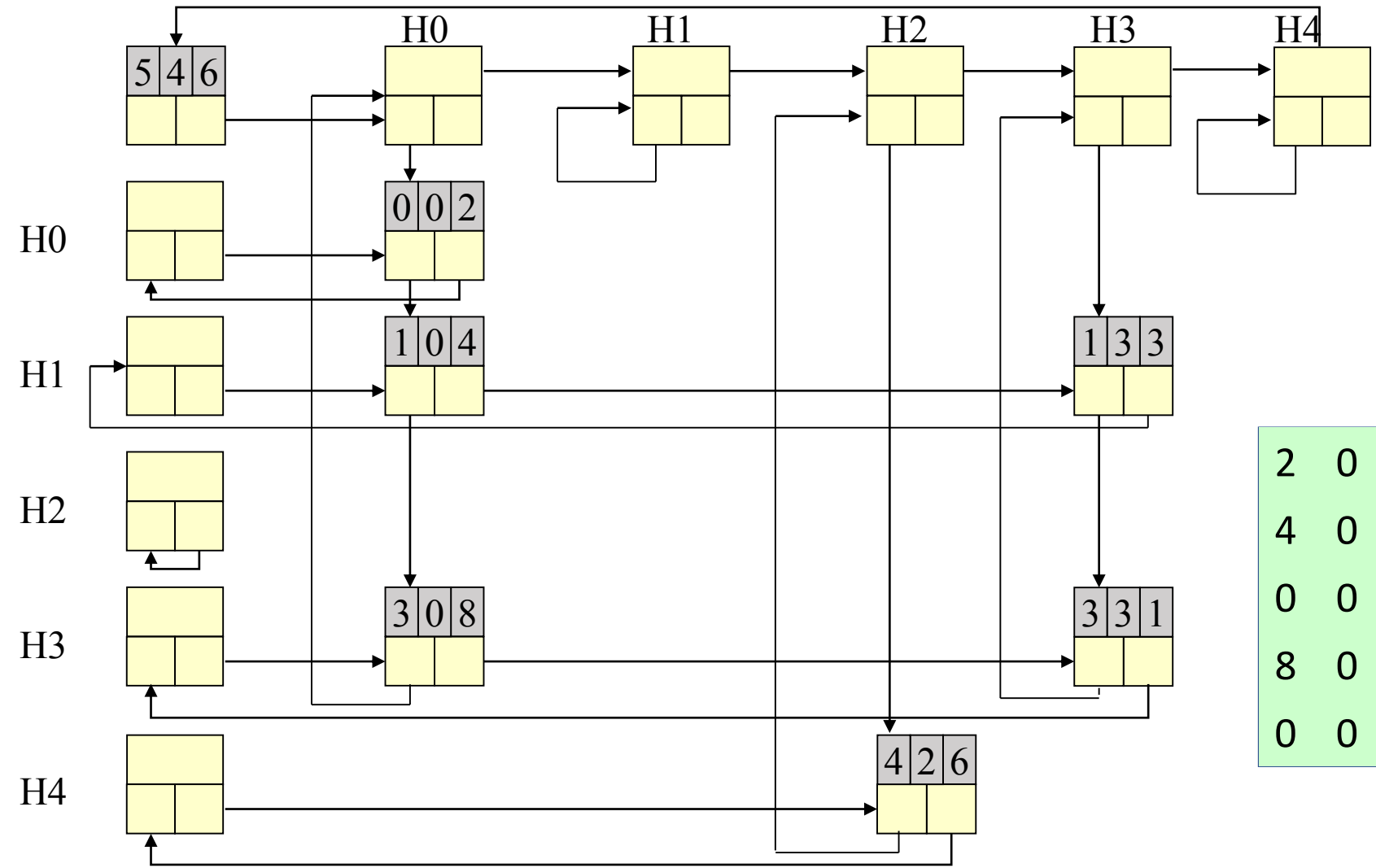


Sparse Matrices – Representation ^{1/2}

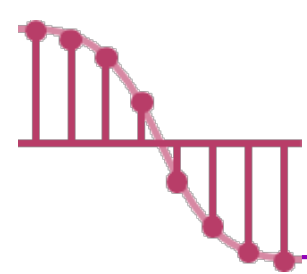
next	
down	right
header node	

row	col	value
down		right

element node

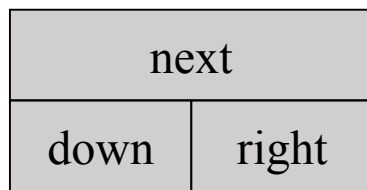


2	0	0	0
4	0	0	3
0	0	0	0
8	0	0	1
0	0	6	0

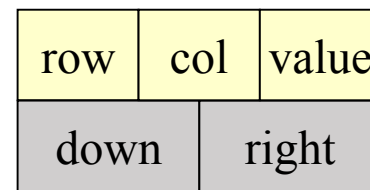


Sparse Matrices – Representation _{2/2}

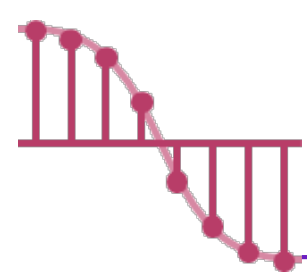
- We represent each column (row) of a sparse matrix as **a circularly linked list** with a header node.
- The header node for row i is also the header node for column i . The number of header nodes is $\max\{\text{numRows}, \text{numCols}\}$.
- Each element node is simultaneously linked into two lists: **a row list, and a column list**.
- Each head node is belonged to three lists: **a row list, a column list, and a header node list**.



header node



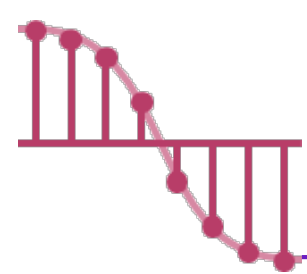
element node



Doubly Linked Lists

- The **problem** of a **singly linked** list:
 - ✓ The only way to **find the node that precedes p** is to **start at the beginning** of the list.
 - ✓ When we have a problem in which it is necessary to move in **either direction**.
- Therefore, it is useful to have **doubly linked lists**.

```
typedef struct node *nodePointer;  
typedef struct node {  
    nodePointer llink;  
    element data;  
    nodePointer rlink;  
};
```

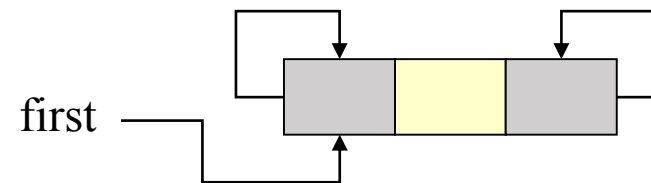
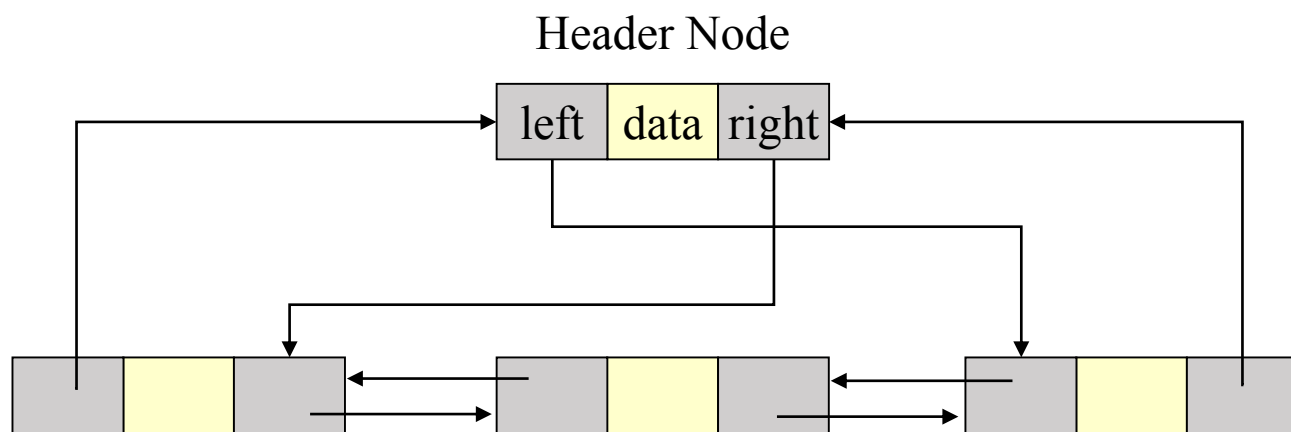



Doubly linked circular list with header node

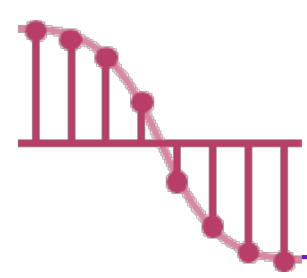
➤ Header node allows us to implement operations more easily.

✓ If **ptr** points to any node in doubly linked list, then

$$\mathbf{ptr = ptr \rightarrow llink \rightarrow rlink = ptr \rightarrow rlink \rightarrow llink}$$



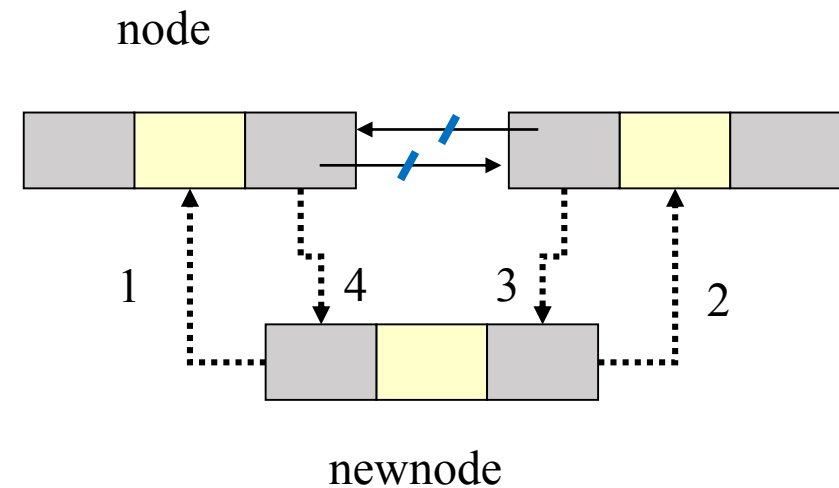
Empty doubly linked circular list with header node

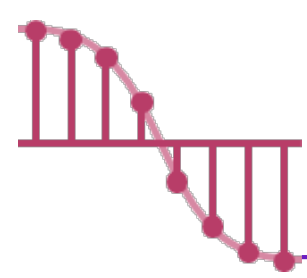


Program 4.26

Insertion into A Doubly Linked Circular List

```
void dinsert( nodePointer node, nodePointer newnode)
{
    /* insert newnode to the right of node */
    newnode→llink = node;          // 1
    newnode→rlink = node→rlink;    // 2
    node→rlink→llink = newnode;    // 3
    node→rlink = newnode;          // 4
}
```





Program 4.27

Deletion from A Doubly Linked Circular List

```
void ddelete (nodePointer node, nodePointer deleted)
{
    /* delete from the doubly linked list */
    if (node == deleted)
        printf("Deletion of header node not permitted. \n");
    else {
        deleted->llink->rlink = deleted->rlink;           // 1
        deleted->rlink->llink = deleted->llink;           // 2
        free(deleted);
    }
}
```

